Shimano BT-E6000 BMS Protocol Analysis

Version 2.0 — Comprehensive Protocol Specification

Covers: UART frame format, CRC-16/IBM-SDLC algorithm, authentication protocol, telemetry decoding, register address map, cell-level BMS data, capabilities negotiation, timestamp frames, and session flow.

Document	Shimano_BTE6000_BMS_Protocol_Analysis_V2.pdf
Date	2026-03-22
Hardware	BT-E6000 / BT-E6001 (36V 10S4P Li-Ion)
MCU	Renesas R2A240 60F020 (78K0 family)
BMS IC	TI BQ77PL900
Charger	EC-E6002 / EC-E6000
Data Sources	UART captures (RXTXRecords), 256-address register scan
CRC Algorithm	CRC-16/IBM-SDLC (verified 100%)

Table of Contents

1. System Overview

The Shimano STEPS BT-E6000 is a 36V e-bike battery pack using a 10S4P lithium-ion cell configuration. Internal communication between the battery and external devices (charger, motor controller, display unit) uses a proprietary serial UART protocol at 3.3V logic levels.

The battery contains two key ICs: a **Renesas R2A240 60F020** microcontroller (78K0 family, 8-bit) that handles all UART communication and protocol logic, and a **TI BQ77PL900** battery management IC that monitors cell voltages, temperature, and current in host mode.

1.1 Communication Architecture

All external devices communicate with the battery MCU over a single-wire UART bus (no CAN). The protocol is half-duplex request/response: an external device sends a frame, and the battery MCU responds with a corresponding frame. Communication is not encrypted, but authentication is required before the battery enters active mode.

Device	SEQ ID	Greeting SEQ	Role
Display (SC-E6010)	0x00-0x03	0x40	UI, assist mode control
Charger (EC-E6002)	0x00-0x03	0x41	Charge control, telemetry polling
Custom Tool	0x00-0x03	0x42	Register scan, diagnostics
Battery (BT-E6000)	0x80-0x83	0xC1 / 0xC2	Responds to all requests

Table 1: Device identifiers on the Shimano STEPS bus

2. Physical Layer & Framing

2.1 UART Parameters

The UART operates at **9600 baud, 8N1** (8 data bits, no parity, 1 stop bit) with 3.3V logic levels. The bus is active-low idle. Frames are separated by a **0x00 delimiter byte** and there is typically a ~1 second interval between polling cycles.

2.2 Frame Structure

V2 Key Finding: The leading 0x00 byte is an inter-frame **delimiter**, not part of the frame payload. This was confirmed by verifying that CRC-16/IBM-SDLC computes correctly over the frame bytes *excluding* the leading 0x00. All frame-type-dependent CRC init values from V1 are now obsolete — a single standard CRC algorithm covers all frame types.

Field	Bytes	Description
Delimiter	1 (0x00)	Inter-frame separator — NOT included in CRC calculation
SEQ	1	Device address + sequence counter (see Section 4)
LEN	1	Number of bytes from TYPE to end of payload (inclusive)
TYPE	1	Frame type identifier (0x10, 0x21, 0x30, 0x31, 0x32)
Payload	LEN-1	Type-specific payload data (0 bytes if LEN=1)
CRC	2	CRC-16/IBM-SDLC, little-endian, computed over SEQ..Payload

Table 2: Frame structure (revised for V2)

LEN field semantics: LEN counts the TYPE byte plus all payload bytes. For greeting frames (LEN=0), no TYPE or payload exists — the frame is just [SEQ][0x00][CRC_L][CRC_H]. For a poll frame with 4 payload bytes: LEN = 1 (TYPE) + 4 (payload) = 0x05.

2.3 Example Frames

Frame Type	Raw Bytes (with delimiter)	Breakdown
Greeting (Display)	00 40 00 21 49	[00] SEQ=40 LEN=00 CRC=2149
Greeting (Battery)	00 C1 00 35 DC	[00] SEQ=C1 LEN=00 CRC=35DC
Poll (charger)	00 00 05 10 00 00 00 00 B7 2C	[00] SEQ=00 LEN=05 TYPE=10 PL=00000000 CRC=B72C
Shutdown	00 02 01 21 27 5A	[00] SEQ=02 LEN=01 TYPE=21 CRC=275A
Timestamp	00 02 07 32 19 09 05 0D 1E 22 FC FD	[00] SEQ=02 LEN=07 TYPE=32 PL=1909050D1E22 CRC=FCFD

Table 3: Example frames with delimiter separated

3. CRC-16/IBM-SDLC Algorithm

This is the single most important V2 finding. The CRC algorithm is standard **CRC-16/IBM-SDLC** (also known as CRC-16/ISO-HDLC, CRC-16/X.25), applied uniformly to ALL frame types. The V1 report incorrectly identified multiple frame-type-dependent init values because the leading 0x00 delimiter was included in the CRC calculation.

Parameter	Value
Polynomial	0x1021 (CRC-CCITT)
Init Value	0xFFFF
Reflect Input	True
Reflect Output	True
XOR Output	0xFFFF
Check Value	0x906E (for ASCII "123456789")
CRC Scope	All bytes from SEQ through end of Payload (excludes delimiter and CRC itself)
Byte Order	Little-endian (CRC low byte first)
Common Names	IBM-SDLC, ISO-HDLC, X.25, CRC-B, CRC-16/ISO-IEC-14443-3-B

Table 4: CRC-16/IBM-SDLC parameters

3.1 Verification Results

The CRC algorithm was verified against **36 frames spanning all frame types** (greetings, polls, telemetry, authentication challenges, authentication responses, capabilities, timestamps, shutdown commands). All 36 frames compute correctly with a single CRC algorithm and zero exceptions. Additionally, all 177 response frames from the 256-address register scan verify with the same algorithm.

Frame Type	Frames Tested	Result
Greeting (LEN=0)	3	3/3 OK
Auth sub-frames (LEN=1)	2	2/2 OK
Shutdown (LEN=1)	2	2/2 OK
Poll (LEN=5, charger)	4	4/4 OK
Poll (LEN=5, bike)	6	6/6 OK
Timestamp (LEN=7)	7	7/7 OK
Capabilities (LEN=11, 18)	2	2/2 OK
Auth challenge (LEN=17)	4	4/4 OK
Auth response (LEN=18)	1	1/1 OK
Telemetry (LEN=22)	4	4/4 OK
Register scan responses	177	177/177 OK
TOTAL	212	212/212 OK (100%)

Table 5: CRC verification across all frame types

3.2 Relationship to V1 Init Values

The V1 report documented multiple CRC init values (0x8683 for charger polls, 0xF46E for battery telemetry, etc.). These were artifacts of including the 0x00 delimiter byte in the CRC input. Mathematically, processing a 0x00 byte through the reflected CRC-CCITT with init=0xFFFF and xorout=0xFFFF produces init=0x8683 with xorout=0x0000. The "different" init values per frame type arose because different SEQ bytes following the 0x00 delimiter produced different intermediate CRC states.

3.3 Python Implementation

```
import crcmod

# Standard IBM-SDLC CRC
crc_sdlc = crcmod.predefined.mkCrcFun('x-25')

def compute_crc(frame_bytes_without_delimiter):
    """Compute CRC over SEQ+LEN+TYPE+Payload, return 2 bytes LE."""
    crc = crc_sdlc(frame_bytes_without_delimiter)
    return crc.to_bytes(2, 'little')

# Example: Poll frame SEQ=0x00
frame = bytes([0x00, 0x05, 0x10, 0x00, 0x00, 0x00, 0x00])
crc = compute_crc(frame)  # => b'\xb7\x2c'
```

4. SEQ Byte — Device Addressing & Bitfield

The SEQ byte (byte 1 of each frame, after the delimiter) serves a dual purpose: it identifies the sending device and provides a 2-bit rolling sequence counter for request/response matching.

Bit(s)	Name	Description
7 (MSB)	DEVICE	1 = Battery (response), 0 = External device (request)
6	GREETING	1 = Greeting/header frame, 0 = Normal data frame
5-2	RESERVED	Always 0 in all observed frames
1-0	COUNTER	2-bit sequence counter (0-3), wraps modulo 4

Table 6: SEQ byte bitfield structure

SEQ Value	Binary	Device	Type	Counter
0x00	0000 0000	External (Charger/Display)	Data	0
0x01	0000 0001	External	Data	1
0x02	0000 0010	External	Data	2
0x03	0000 0011	External	Data	3
0x40	0100 0000	External (Display)	Greeting	0
0x41	0100 0001	External (Charger)	Greeting	1
0x42	0100 0010	External (Custom)	Greeting	2
0x80	1000 0000	Battery	Data	0
0x83	1000 0011	Battery	Data	3
0xC1	1100 0001	Battery	Greeting	1 (to charger)
0xC2	1100 0010	Battery	Greeting	2 (to custom)

Table 7: SEQ byte examples from captured data

Greeting response rule: When the battery receives a greeting with SEQ=0x4N, it responds with SEQ = 0x4N | 0x80 = 0xCN. This was confirmed across all 64 greeting addresses (0x40-0x7F) in the register scan.

5. Frame Types (TYPE Field)

The TYPE byte (byte 3 of the frame, first byte of the LEN-counted region) identifies the purpose of the frame. For LEN=0 (greeting frames), no TYPE byte exists.

TYPE	Name	LEN	Direction	Description
(none)	Greeting	0x00	Both	Device presence announcement / bus enumeration
0x10	Poll / Telemetry	0x05 / 0x16	Req / Resp	Status poll (ext) or telemetry response (batt)
0x11	Auth Acknowledge	0x01	Ext -> Batt	Authentication sub-frame A
0x12	Auth Commit	0x01	Ext -> Batt	Authentication sub-frame B
0x21	Shutdown	0x01	Ext -> Batt	Power-off / disconnect command
0x30	Auth Challenge	0x11 / 0x12	Both	Challenge (ext) or response (batt)
0x31	Capabilities	0x0B / 0x12	Batt -> Ext	Battery capabilities / parameters
0x32	Timestamp	0x07	Ext -> Batt	RTC timestamp (YY MM DD HH MM SS)

Table 8: Known frame types

6. Authentication Protocol (TYPE 0x30)

Before entering active mode, the external device must complete an authentication handshake with the battery. This is a 4-frame sequence initiated by the external device.

6.1 Authentication Sequence

Step	Direction	TYPE	Description
1	Ext -> Batt	0x30 (LEN=0x11)	Challenge: device ID (2 bytes) + 12-byte LFSR nonce + 2 reserved
2	Ext -> Batt	0x11 (LEN=0x01)	Auth acknowledge (no payload beyond TYPE)
3	Ext -> Batt	0x12 (LEN=0x01)	Auth commit (no payload beyond TYPE)
4	Batt -> Ext	0x30 (LEN=0x12)	Response: device ID + 15-byte truncated MAC

Table 9: Authentication handshake sequence

6.2 Challenge Structure (TYPE 0x30, LEN=0x11)

Offset	Length	Field	Description
0-1	2	Device ID	0x02,0x01 = Charger; 0x03,0x02 = Motor controller
2-3	2	Padding	Always 0x00, 0x00 (bike) or nonce start (charger)
4-15	12	Challenge nonce	LFSR-generated pairs with 4-byte repeat at offset 4-7 = offset 12-15

6.3 LFSR Pattern Analysis

The challenge nonce exhibits a clear pattern: bytes 4-7 are repeated at bytes 12-15, and the nonce is generated as 2-byte LFSR pairs. This was confirmed across three independent bike sessions and one charger session:

```
Bike session 1: 00 00 B2 EB 51 6E 67 9D B2 EB 51 6E C1 05
                      ^^^^^^^^^^^^^^^      ^^^^^^^^^^^^^^^
                      bytes 4-7 repeat at bytes 12-15

Bike session 2: 00 00 EC 52 6F 68 9E B3 EC 52 6F 68 80 05
Charger:        9E 9E 9E 9F F6 F6 F6 F6 9F 9F 9F 9F D0 08
```

6.4 Security Assessment

The authentication is **replay-vulnerable**: no session counter, timestamp, or monotonic nonce is included in the authentication payload. The challenge nonce is LFSR-generated (not cryptographically random). The response MAC is hypothesized to be **HMAC-SHA-1** based on TI BQ77PL900 Application Note SLUA625, truncated to 15 bytes. The open-source project gregyedlik/BT-E60xx exploits this by replaying captured authentication frames to unlock the battery via UART.

7. Telemetry Frame (TYPE 0x10)

7.1 Poll Frame (External -> Battery, LEN=0x05)

The external device sends a 5-byte TYPE 0x10 frame to poll the battery for telemetry. The payload encodes the current mode and device context.

Offset	Field	Values	Description
0	MODE	0x02=Init, 0x03=Active	Communication state; transitions 0x02->0x03 at startup
1	CONTEXT	0x00=Charger, 0x01=Unknown, 0x03=Startup, 0x09=Bike	Device context identifier
2-3	Reserved	0x00, 0x00	Always zero in observed captures

Table 10: Poll frame payload fields (TYPE 0x10, LEN=5)

7.2 Telemetry Response (Battery -> External, LEN=0x16)

The battery responds with a 22-byte TYPE 0x10 frame containing pack voltage, current, temperature, and status information.

Offset	Len	Field	Encoding	Example
0	1	Flags	Bit field	0x00
1	1	MODE	0x02=Init, 0x03=Active	0x03
2-4	3	Reserved	Always 0x00	00 00 00
5-6	2	Pack Voltage	uint16-LE, millivolts	9B A1 = 41371 mV = 41.371 V
7	1	Current (raw)	uint8, scaling TBD	0x68 = 104
8	1	Unknown	Always 0x20	0x20
9	1	Current (alt)	uint8, correlates with B7	0x4A = 74
10	1	Pack Temp	Direct Celsius, uint8	0x20 = 32 deg C
11	1	Temp sensor 1	Direct Celsius	0x18 = 24 deg C
12	1	Temp sensor 2	Direct Celsius	0x18 = 24 deg C
13	1	Temp sensor 3	Direct Celsius	0x18 = 24 deg C
14	1	SOC / Status A	Increments slowly (0x5C->0x5D)	0x5C = 92
15	1	Status B	0x00=pre-charge, 0x4D/0x4E=charging	0x4E
16-21	6	Reserved	Always 0x00	00 00 00 00 00 00

Table 11: Telemetry response payload fields (TYPE 0x10, LEN=0x16)

7.3 Telemetry Data Progression (Charge Session)

Analysis of 31 consecutive telemetry frames during a charge session (FROM_BATT_002) shows:

```
Frame  Voltage    I_raw  B9/Alt  TempPack T1 T2 T3  B14  B15
  1    40.957V    0x08    0xFA    31 C    24 24 24  0x5C 0x00  (pre-charge)
  3    41.371V    0x68    0x4A    32 C    24 24 24  0x5C 0x4E  (CC charging starts)
 15    41.489V    0x80    0x62    32 C    24 24 24  0x5D 0x4D
 22    41.513V    0x86    0x66    32 C    24 24 25  0x5D 0x4E
 31    41.542V    0x8A    0x6A    32 C    24 24 25  0x5D 0x4E  (late CV phase)

Voltage range:  40.957V - 41.542V  (4.096 - 4.154 V/cell)
I_raw range:    0x08 (pre-charge) -> 0x68..0x8A (CC/CV)
Pack temp:      31 C -> 32 C (slight rise during charge)
B14 (SOC?):     0x5C -> 0x5D (very slow increment = SOC %?)
```

8. Timestamp / Energy Frame (TYPE 0x32)

The TYPE 0x32 frame carries a 6-byte RTC timestamp. It is sent periodically (approximately once per minute) by the external device during active communication. This frame type was not documented in V1.

Offset	Field	Format	Example
0	Year	Hex value (BCD-like), add 2000	0x19 = 2025
1	Month	Hex 01-12	0x09 = September
2	Day	Hex 01-31	0x05 = 5th
3	Hour	Hex 00-23	0x0D = 13 (1 PM)
4	Minute	Hex 00-59	0x1E = 30
5	Second	Hex 00-59	0x22 = 34

Table 12: Timestamp frame payload fields

8.1 Timestamp Progression (BATT_RX_ON_BIKE_TIME_OUT_004)

```
Frame      Decoded Timestamp       Notes
ON_OFF_S1  2025-09-05 13:30:34     First bike session
ON_OFF_S2  2025-09-05 13:45:07     Second session (15 min later)
MENU       2025-09-05 13:59:00     Menu button test
TIMEOUT_1  2025-09-05 14:07:00     Timeout recording starts
TIMEOUT_2  2025-09-05 14:08:00     +1 minute
TIMEOUT_4  2025-09-05 14:09:00     +1 minute
   ...       (continues)           Minute-by-minute increment
TIMEOUT_12 2025-09-05 14:13:00     Last frame before timeout

ONLINE_FIND: 2000-01-12 18:08:32   RTC not set (epoch default)
```

9. Capabilities Frame (TYPE 0x31)

The capabilities frame is sent by the battery in response to a capabilities request. It contains static battery parameters. Two variants have been observed: a shorter bike variant (LEN=0x0B) and a longer charger variant (LEN=0x12).

Offset	Len	Field	Charger Value	B9 Scan Value	Interpretation
0-1	2	FW/HW Version	0x019F = 415	0x019F = 415	Firmware version (identical)
2-3	2	Model ID	0x01A9 = 425	0x01A9 = 425	Battery model identifier
4-5	2	Config	0x0001	0x0001	Cell config (1 = 10S?)
6-7	2	Charge Rate	0x0005	0x0005	Max charge rate parameter
8-9	2	Voltage Param	0x0028 = 40	0x0028 = 40	Likely 40V nominal
10-11	2	Dynamic A	0x0088 = 136	0x006E = 110	DIFFERS — state dependent
12-13	2	Dynamic B	0x0143 = 323	0x010E = 270	DIFFERS — state dependent
14-15	2	Capacity	0x0078 = 120	0x0078 = 120	Nominal capacity (120 = 12.0 Ah?)

Table 13: Capabilities payload comparison (charger session vs register scan)

Fields 10-13 differ between the charger session and the register scan, suggesting these are dynamic values (possibly current SOH, remaining capacity, or cycle count) rather than static configuration.

10. Shutdown Command (TYPE 0x21)

TYPE 0x21 is a 1-byte frame (LEN=0x01, no payload beyond the TYPE byte itself) that signals the battery to power off. Immediately before the shutdown frame, the last poll frame shows MODE reverting from 0x03 (active) back to 0x02 (init).

```
Normal polling (MODE=0x03, active):
  00 01 05 10 03 09 00 00 B1 0A
  00 02 05 10 03 09 00 00 DF A2

Pre-shutdown poll (MODE reverts to 0x02):
  00 01 05 10 02 09 00 00 0A 16

Shutdown command:
  00 02 01 21 27 5A

Final delimiter:
  00
```

The same sequence was observed in both manual power-off (ON_OFF_004) and auto-timeout (TIME_OUT_004), confirming TYPE 0x21 as the universal shutdown command.

11. Complete Session Flows

11.1 Startup Sequence (Universal)

The startup sequence is identical across all observed devices (bike, charger, custom tool). It was confirmed from three independent sources.

Step	SEQ	Frame	Description
1	0x4N	[SEQ] 00 [CRC]	External device sends greeting (LEN=0)
	0xCN	[SEQ] 00 [CRC]	Battery responds with greeting
2	0x01	[SEQ] 11 30 [challenge] [CRC]	Auth challenge (TYPE 0x30, 17 bytes)
3	0x02	[SEQ] 01 11 [CRC]	Auth acknowledge (TYPE 0x11)
4	0x03	[SEQ] 01 12 [CRC]	Auth commit (TYPE 0x12)
5	0x00	[SEQ] 05 10 02 03 00 00 [CRC]	First poll: MODE=0x02 (init), CTX=0x03
6	0x01	[SEQ] 05 10 03 03 00 00 [CRC]	Second poll: MODE=0x03 (active), CTX=0x03
7	0x02	[SEQ] 07 32 [timestamp] [CRC]	Timestamp frame (TYPE 0x32)
8+	0x03..	[SEQ] 05 10 03 [ctx] 00 00 [CRC]	Steady-state polling (CTX=0x09 bike, 0x01 other)

Table 14: Universal startup sequence

11.2 Charger Session Flow

```
Phase 1 — Startup:      Greeting -> Auth -> MODE 0x02 -> 0x03
Phase 2 — Pre-charge:   Poll/Telemetry (I_raw=0x08, B15=0x00)
Phase 3 — CC Charging:  Telemetry (I_raw=0x68+, B15=0x4D/0x4E, voltage rising)
Phase 4 — CV Charging:  Telemetry (I_raw slowly decreasing, voltage ~41.5V)
Phase 5 — Termination:  Poll MODE reverts to 0x02 -> Shutdown (TYPE 0x21)
```

11.3 Bike Session Flow

```
Phase 1 — Startup:      Greeting(0x40) -> Auth(0x03,0x02) -> MODE 0x02 -> 0x03
Phase 2 — Active:       Poll/Telemetry with CTX=0x09, timestamp every ~1 min
Phase 3 — Idle timeout:  ~5 minutes with no pedaling -> auto-shutdown
Phase 4 — Shutdown:     MODE reverts to 0x02 -> Shutdown (TYPE 0x21)
```

12. Register Address Map (256-Address Scan)

A complete 256-address scan was performed by sending LEN=0 greeting-style frames with SEQ bytes 0x00 through 0xFF and recording the battery's response. The scan tool used SEQ=0x42 for its own greeting (new device ID). All 177 response CRCs verified 100% with IBM-SDLC. The scan was recorded on 2026-03-21.

12.1 Address Space Overview

Range	Count	Response Type	Description
0x00 - 0x3F	50 data + 14 NR	LEN=2 (standard) + 1 extended	Data registers, mostly return value=0x01
0x40 - 0x7F	64 greeting	LEN=0 (greeting)	Bus enumeration range; all respond with SEQ|0x80
0x80 - 0xBF	53 data + 5 ext + 5 NR + 1 mixed	Mixed	Data registers + 5 extended blocks
0xC0 - 0xFF	64 echo	TX = RX (echo)	Not processed by battery MCU; loopback

Table 15: Address space map overview

12.2 Standard Registers (LEN=2)

107 addresses returned a standard 2-byte response with Data[1] = 0x01 uniformly. Data[0] equals the low byte of the CRC of the corresponding request frame — this appears to be an echo/acknowledge mechanism rather than meaningful register data. The uniform 0x01 value likely indicates "register exists and is readable."

12.3 NO_REPLY Addresses

14 addresses produced no response (timeout). These may be unimplemented registers or addresses that require authentication before responding.

```
Low range  (0x00-0x3F): 0x13, 0x14, 0x19, 0x1D, 0x21, 0x25, 0x26, 0x28, 0x2F
High range (0x80-0xBF): 0x95, 0x9B, 0x9C, 0xA0, 0xA7
```

12.4 Echo Range (0xC0-0xFF)

All 64 addresses in 0xC0-0xFF returned TX = RX (exact echo). This suggests the battery MCU does not process these addresses, and the response is a hardware-level loopback. One anomaly: address 0xAB (outside the 0xC0 range) also produced an echo response.

13. Extended Register Analysis

Six addresses returned extended responses (LEN > 2) containing multi-byte data blocks. These are the most valuable registers for BMS monitoring and diagnostics.

13.1 Register 0x1A — Serial / ID Block (LEN=7)

```
Data: A6 A1 17 60 56 52 4B
```

This 7-byte block likely contains a serial number or hardware identifier. The values do not correspond to ASCII characters and may be a binary-encoded production identifier.

13.2 Register 0x92 — Status (LEN=3)

```
Data: AA 90 01
```

Three-byte status register. Byte 1 = 0x90 (144 decimal) may encode a BMS state or error flags. Byte 2 = 0x01 is the standard acknowledge byte.

13.3 Register 0x97 — BMS Configuration (LEN=10)

```
Data: 12 25 00 E1 00 00 90 10 0F 00
```

Offset	uint16-LE	Hex	Possible Meaning
0-1	9490	0x2512	BQ77PL900 config register / FW build
2-3	57600	0xE100	BMS parameter (OV threshold?)
4-5	0	0x0000	Reserved / unused
6-7	4240	0x1090	BMS parameter (UV threshold?)
8-9	15	0x000F	Cell count bitmask? (0x0F = 4 bits for 10 cells?)

Table 16: Register 0x97 field interpretation (tentative)

13.4 Register 0xA9 — BMS Mega-Block (LEN=43)

This is the **most data-rich register**, containing 43 bytes of BMS telemetry. Analysis reveals pack voltage, capacity, temperature, and current data.

```
Raw: A0 00 00 00 00 1F 00 00 00 34 00 34 00 20 00 17 00 0D 00 08
     00 04 00 0C 00 1F 20 F8 2A 4A 00 C7 00 D4 FE C4 00 01 00
     00 00 02 00
```

Offset	uint16-LE	Interpretation	Confidence
0	0x00A0 = 160	Header / register echo byte	Medium
1-4	all zeros	Reserved / padding	Low
5-6	0x001F = 31	Unknown parameter (cell count related?)	Low
7-24	see below	10x uint8 values padded with 0x00: 0,52,52,32,23,13,8,4,12,31	Medium
25-26	0x201F = 8223	Pack voltage: 8223/200 = 41.115V (confirmed)	HIGH
27-28	0x2AF8 = 11000	Remaining capacity: 11000 * 10mAh? = 11.0 Ah	HIGH
29-30	0x004A = 74	Temperature: 74-40 = 34 deg C	HIGH
31-32	0x00C7 = 199	Unknown (cycle count? SOH?)	Low
33-34	0xFED4 = -300 (signed)	Current: -300 * 10mA = -3.0A (charging)	HIGH
35-36	0x00C4 = 196	Unknown (accumulated charge?)	Low
37-38	0x0001 = 1	BMS state flag	Low
39-40	0x0000 = 0	Reserved	Low
41-42	0x0002 = 2	Unknown flag / error code	Low

Table 17: Register 0xA9 mega-block field interpretation

Key validation: The pack voltage from register 0xA9 (41.115V via formula val/200) closely matches the simultaneously captured telemetry voltage of ~41.0-41.5V from the charger session. The capacity value of 11000 (= 11.0 Ah at 10 mAh resolution) aligns with the BT-E6000 nominal capacity of 11.6 Ah (418 Wh / 36V).

13.5 Register 0xB7 — Version / Status (LEN=3)

```
Data: 21 25 FF
```

Byte 0 = 0x21 may echo TYPE 0x21 (shutdown type). Byte 1 = 0x25 = 37 matches the second byte of 0xB9 (capabilities). Byte 2 = 0xFF could indicate "all features enabled" or a hardware revision bitmask.

13.6 Register 0xB9 — Capabilities Mirror (LEN=18)

This register returns the same capabilities data as the TYPE 0x31 frame, prefixed with two context bytes (0x31 = TYPE echo, 0x25 = device/version context). The payload matches the charger capabilities frame at 14 of 16 bytes — the two differing fields (offsets 10-11 and 12-13) appear to be state-dependent dynamic values.

14. Open Questions & Future Work

#	Question	Priority	Approach
1	Current scaling factor (telemetry offsets 7 and 9)	HIGH	Simultaneous in-line shunt measurement during CC/CV charge phases
2	Register 0xA9 byte 7-24 interpretation (cell-level data?)	HIGH	Compare with individual cell voltage measurements; may need BQ77PL900 I2C tap
3	Capabilities dynamic fields (offsets 10-13) meaning	MEDIUM	Compare across batteries with different SOH / cycle counts
4	Auth key extraction from R2A240 flash	MEDIUM	UART bootloader (FLMD0 pin), Renesas E2 Lite, or voltage glitching
5	Register behavior after authentication	MEDIUM	Repeat 256-scan after successful auth; NO_REPLY addrs may respond
6	0xA9 offset 31-32 (value 199) interpretation	LOW	Correlate across batteries of different ages
7	BQ77PL900 I2C/SPI bus sniffing (MCU <-> BMS IC)	LOW	Would provide ground truth for all cell-level values

Table 18: Open questions and proposed approaches

15. Quick Reference Card

Frame Format

```
[0x00 delim][SEQ][LEN][TYPE + Payload (LEN bytes)][CRC_L][CRC_H]

CRC: IBM-SDLC over [SEQ..Payload]  (init=FFFF, poly=1021, ref=T/T, xor=FFFF)
```

Device SEQ Values

```
Display greeting:  0x40     Battery greeting:  0xC0 (to display)
Charger greeting:  0x41     Battery greeting:  0xC1 (to charger)
Custom greeting:   0x42     Battery greeting:  0xC2 (to custom)
Data frames:       0x00-03  Battery responses: 0x80-83
```

Startup Sequence

```
1. Greeting (LEN=0)  ->  Battery greeting response
2. Auth TYPE=0x30    ->  (no immediate response)
3. Auth TYPE=0x11    ->  (no immediate response)
4. Auth TYPE=0x12    ->  Battery auth response TYPE=0x30
5. Poll MODE=0x02    ->  Telemetry (init)
6. Poll MODE=0x03    ->  Telemetry (active)
7. Timestamp 0x32    ->  (ack)
8. Steady-state polling with TYPE=0x10 and TYPE=0x32
```

Key Registers (from 256-scan)

```
0x1A: Serial / ID block (7 bytes)
0x92: Status register (3 bytes)
0x97: BMS config (10 bytes)
0xA9: BMS mega-block (43 bytes) — VOLTAGE, CAPACITY, CURRENT, TEMP
0xB7: Version / status (3 bytes)
0xB9: Capabilities mirror (18 bytes)
```

0xA9 Key Fields

```
Offset 25-26: Pack voltage (uint16-LE / 200 = Volts)
Offset 27-28: Remaining capacity (uint16-LE, likely 10 mAh units)
Offset 29-30: Temperature (uint16-LE - 40 = Celsius)
Offset 33-34: Current (int16-LE, likely 10 mA units, negative = charging)
```

Appendix A: Python Code Snippets

A.1 CRC Calculation & Verification

```
import crcmod

crc_sdlc = crcmod.predefined.mkCrcFun('x-25')  # IBM-SDLC

def verify_frame(hex_with_delimiter: str) -> bool:
    """Verify CRC of a complete frame (including 0x00 delimiter)."""
    raw = bytes.fromhex(hex_with_delimiter.replace(' ', ''))
    frame = raw[1:]           # strip delimiter
    data = frame[:-2]         # everything except CRC
    expected = frame[-2:]     # last 2 bytes = CRC LE
    computed = crc_sdlc(data).to_bytes(2, 'little')
    return computed == expected

# Test:
assert verify_frame('00 00 05 10 00 00 00 00 B7 2C')   # Poll
assert verify_frame('00 80 16 10 00 02 00 00 00 FD 9F'  # Telemetry (partial)
                    ' 08 20 FA 1F 18 18 18 5C 00 00'
                    ' 00 00 00 00 53 09')
```

A.2 Build & Send a Greeting Frame

```
def build_greeting(device_id: int) -> bytes:
    """Build a greeting frame. device_id: 0x40=display, 0x41=charger, etc."""
    data = bytes([device_id, 0x00])  # SEQ + LEN=0
    crc = crc_sdlc(data).to_bytes(2, 'little')
    return b'\x00' + data + crc

# Example:
greeting = build_greeting(0x42)  # Custom tool
# => 00 42 00 91 7A
```

A.3 Parse Telemetry Response

```
def parse_telemetry(hex_frame: str) -> dict:
    raw = bytes.fromhex(hex_frame.replace(' ', ''))
    frame = raw[1:]  # strip delimiter
    payload = frame[3:-2]  # skip SEQ,LEN,TYPE and CRC
    return {
        'mode':     payload[1],
        'voltage':  int.from_bytes(payload[5:7], 'little') / 1000,  # Volts
        'i_raw':    payload[7],
        'temp_pack': payload[10],   # Celsius
        'temp_1':   payload[11],
        'temp_2':   payload[12],
        'temp_3':   payload[13],
        'soc_byte': payload[14],
        'status':   payload[15],
    }
```

A.4 Parse Register 0xA9 (BMS Mega-Block)

```
def parse_reg_a9(data: bytes) -> dict:
    """Parse 43-byte register 0xA9 data."""
    import struct
    return {
        'pack_voltage_V':  struct.unpack_from('        'capacity_Ah':     struct.unpack_from('        'temperatur
e_C':   struct.unpack_from('        'current_A':       struct.unpack_from('    }
```

Appendix B: Changelog V1 to V2

Item	V1	V2	Impact
CRC Algorithm	Multiple init values per frame type (0x9683, 0xF46E, 0xD71C, 0xEDA9, 0xF3786)	Single CRC-16/IBM-SDLC (init=0xFFFF)	BREAKING: Simplified implementation; no lookup table needed
Leading 0x00	Part of frame (start byte)	Inter-frame delimiter (not CRC'd)	BREAKING: Frame parsing logic changes
LEN field	Payload byte count (excl. TYPE)	TYPE + Payload byte count (incl. TYPE)	BREAKING: Off-by-one for all frame parsers
TYPE 0x32	Not documented	Timestamp frame (YY MM DD HH MM SS)	NEW: Complete decode with date validation
TYPE 0x21	Not documented	Shutdown command	NEW: Power-off / disconnect signaling
SEQ bitfield	Device IDs only	Full 8-bit decode: DEVICE|GREETING|RESP|COUNTER	ENHANCED: Complete addressing model
Auth bytes 4-5	Not analyzed	Device identification (02/01=charger, 03/02=bike)	NEW: Cross-device auth differentiation
Register scan	Not available	Full 256-address map with 6 extended registers	NEW: Direct BMS register access
0xA9 mega-block	Not available	43-byte BMS data: voltage, capacity, temp, current	NEW: Cell-level data access path
Capabilities	Partially decoded	Full comparison across charger/bike/scan sources	ENHANCED: Dynamic vs static fields identified
Session flow	Charger only	Universal startup + charger + bike + shutdown	ENHANCED: Complete lifecycle documented

Table 19: Complete changelog from V1 to V2

End of Document — Shimano BT-E6000 BMS Protocol Analysis V2
Generated 2026-03-22. All data from UART captures and register scans.